Global Stat Solutions

Bid Recommendation System

Expected Value Optimization for Commercial Real Estate Appraisal Bid Pricing

Technical Report

Authors: Global Stat Solutions

Version: 2.0

Date: February 2026

Contents

1 Executive Summary

Commercial real estate appraisal firms face a fundamental pricing dilemma on every engagement: bid too high and lose the job; bid too low and win but leave revenue on the table. The **Bid Recommendation System** solves this with a dual-model machine learning approach that jointly optimizes bid pricing and win probability through **Expected Value (EV) maximization**.

The system comprises two independent LightGBM models operating in tandem:

- **Phase 1A (Regression):** Predicts the optimal bid fee given market context — "What should we charge?"
- **Phase 1B (Classification):** Predicts the probability of winning at that fee — "Will we win at this price?"

The combined metric EV = $P(\text{Win}) \times \text{BidFee}$ drives recommendations. For example, a \$5,200 bid with 25% win probability (EV = \$1,300) is objectively worse than a \$3,500 bid with 65% win probability (EV = \$2,275). The system recommends the latter.

Key results:

- Bid Fee model: $R^2 = 0.972$, RMSE = \$354, Median AE = \$35
- Win Probability model: AUC-ROC = 0.870, Accuracy = 79.0%, Overfitting ratio = 1.09×
- Win probability model trained *with* BidFee as a feature — learns fee-sensitivity directly from data
- Data integrity: removed leaky job-derived features (JobCount, IECount, etc.) that inflated prior metrics
- LightGBM outperforms XGBoost by 12.8% (bid fee) and econometric panel models by >85% on RMSE
- 68 carefully engineered features for bid fee, 72 for win probability, reduced from 84 via SHAP-based selection

The system is deployed as a full-stack web application: a React frontend on Vercel, a Flask API on Render, with LightGBM models in portable text format.

2 Problem Statement and Business Context

2.1 The Bid Pricing Challenge

In commercial real estate appraisal, firms submit competitive bids to win engagement contracts. Each bid represents a one-shot pricing decision with asymmetric consequences:

- **Overbidding:** Lose the engagement entirely — zero revenue.
- **Underbidding:** Win the engagement but at a suboptimal margin — reduced profitability.
- **Optimal bidding:** Maximize *expected* revenue across the portfolio.

Historically, bid pricing relied on appraiser judgment, historical norms, and ad hoc spreadsheet lookups. This approach suffers from:

1. **Inconsistency:** Different offices and appraisers price similar engagements differently.
2. **No win probability estimation:** Firms had no systematic way to estimate the likelihood of winning at a given price.
3. **No portfolio optimization:** Individual bids were priced in isolation, not in the context of expected revenue across all active bids.

2.2 Expected Value Framework

The system reframes bid pricing as an optimization problem:

$$\text{EV}(\text{fee}) = P(\text{Win} \mid \text{fee}, \text{context}) \times \text{fee} \tag{1}$$

This formulation captures the fundamental trade-off: higher fees increase per-deal revenue but decrease win probability. The optimal bid maximizes expected revenue. By combining predictions from both models, the system provides a principled recommendation that balances competitiveness with profitability.

3 Data Overview

3.1 Dataset Description

The dataset comprises bid records from a national commercial real estate appraisal firm spanning January 2018 through December 2025. After preprocessing, the system uses data from **January 2023 onward** (52,308 records) based on empirical evidence that a market regime shift makes older data counterproductive for generalization (see Section 9.1).

Table 1: Dataset Summary Statistics

Attribute	**Value**
Total records (raw)	165,898
Records after preprocessing	114,287
Records used for training (2023+)	52,308
Training set (60%)	31,384
Validation set (20%)	10,462
Test set (20%)	10,462
Overall win rate	48.7%
Mean BidFee (2023+)	\$3,363
Median BidFee (2023+)	\$3,000
Standard deviation of BidFee	\$2,131
Unique offices	50+
Unique states	50
Unique property types	15+

3.2 Target Variable Characteristics

The target variable `BidFee` exhibits right-skewed distribution with a mean of $3,363 and a long tail of high-value engagements. Extreme outliers (up to $100M in raw data) were capped at the 99th percentile ($15,000) during preprocessing.

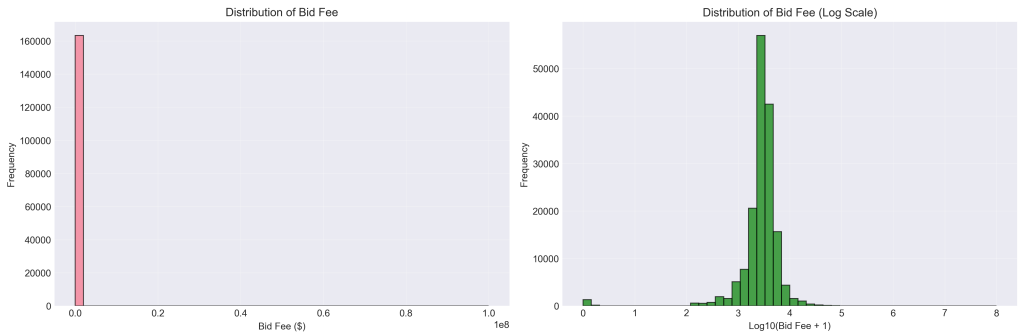


Figure 1: Distribution of BidFee values after preprocessing, showing the right-skewed nature of engagement pricing.

3.3 Business Segment Distribution

The dataset is dominated by the **Financing** segment, which accounts for the vast majority of bid records. Understanding this distribution is critical because the model's accuracy varies by segment data availability.

Table 2: Business Segment Distribution

Business Segment	Record Count	Percentage
Financing	92,945	81.2%
Unknown	12,116	10.6%
Litigation	3,128	2.7%
Due Diligence	2,481	2.2%
Tax	1,574	1.4%
Insurance	1,163	1.0%
Other segments	<1% each	0.9%

Segments with more data naturally produce more confident predictions. The confidence scoring system (Section 8) explicitly accounts for segment sample size.

3.4 Geographic Distribution

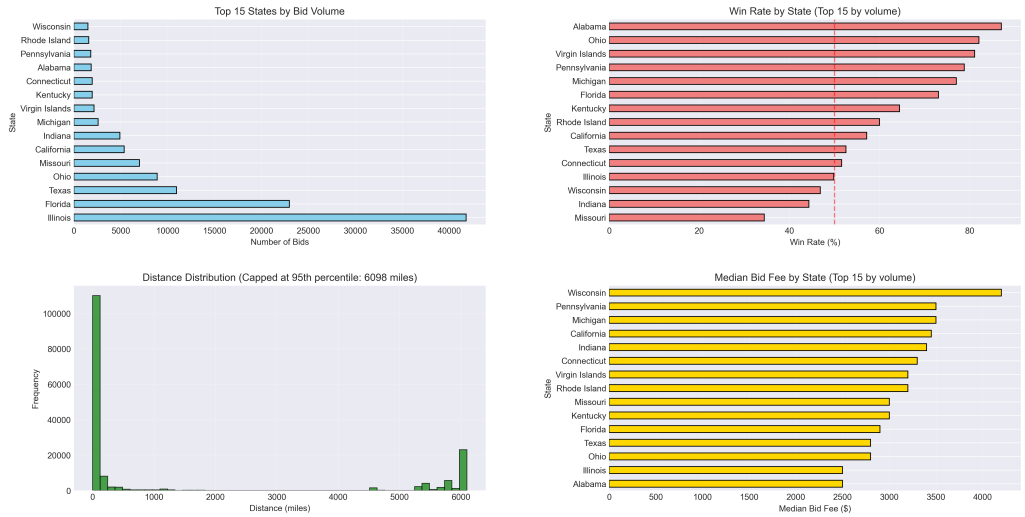


Figure 2: Geographic distribution of bid records across U.S. states. Illinois, Florida, and Texas are the top three states by volume.

The geographic concentration in Illinois (25%), Florida (14%), and Texas (7%) affects model confidence: states with thousands of historical bids produce far more reliable predictions than states with fewer than 50 records.

3.5 Temporal Patterns

Figure 3: Time series analysis of bid volumes and fee trends, showing seasonal patterns and the 2023 regime shift.

4 System Architecture

4.1 High-Level Architecture

The Bid Recommendation System is a full-stack application with three layers: a React frontend, a Flask REST API, and an offline ML training pipeline.

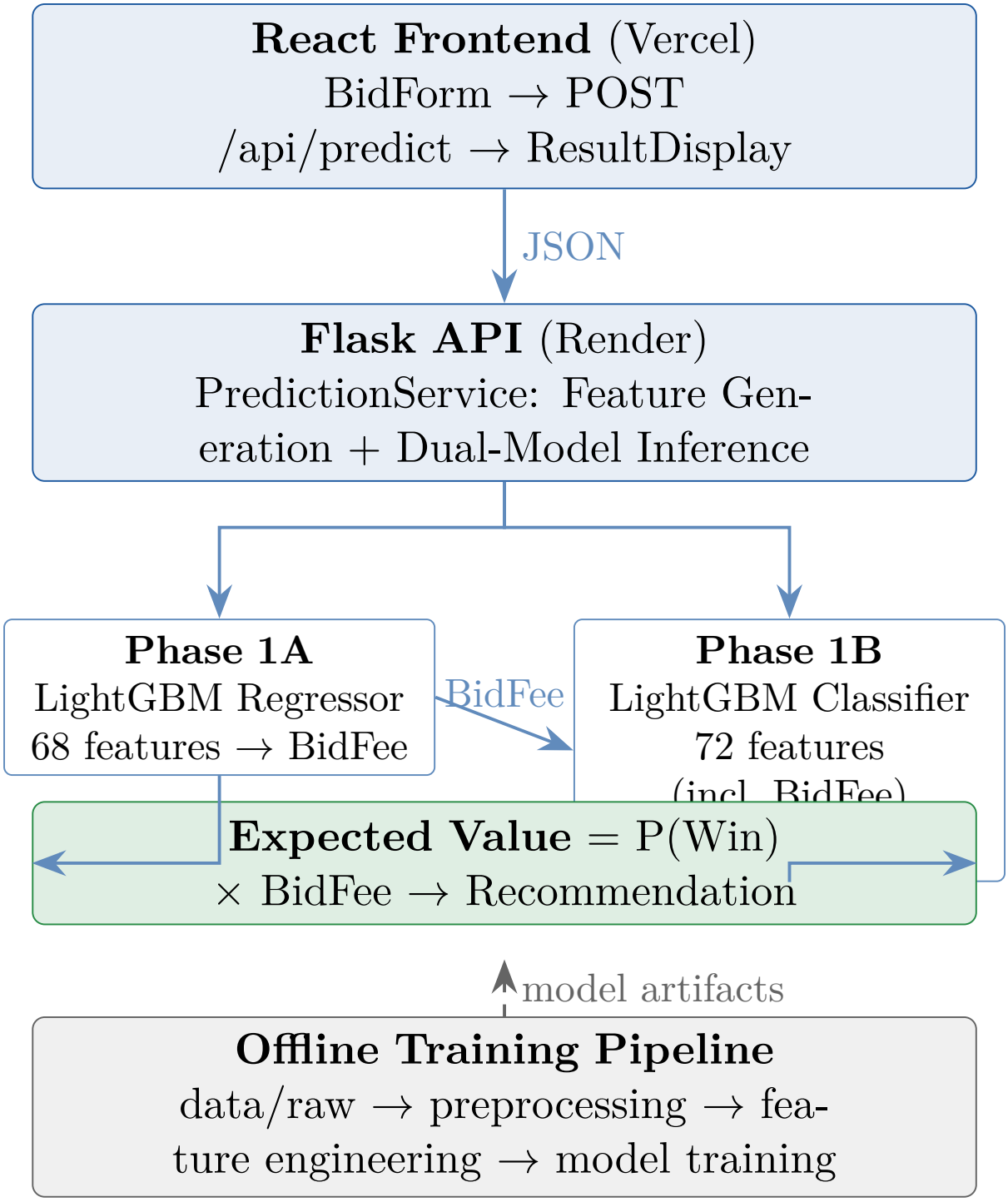


Figure 4: System architecture: end-to-end flow from user input to bid recommendation.

4.2 Component Details

React Frontend (Vercel)
Interactive web interface with a bid input form (6 fields: Business Segment, Property Type, State, Office, Target Time, Distance) and a results display showing predicted fee, confidence interval, win probability, expected value, and actionable recommendation.

Flask API (Render)
RESTful API that receives bid parameters, generates engineered features from 6 raw inputs using precomputed statistics, runs dual-model inference (injecting the predicted fee into the win probability model), and returns a structured JSON response.

ML Training Pipeline (Offline)
Sequential scripts that process raw bid data through cleaning, feature engineering (68 features across 9 categories), and model training. Outputs portable LightGBM `.txt` model files and metadata JSON for deployment.

Configuration Layer
Centralized `model_config.py` with all hyperparameters, paths, feature exclusions, data filtering dates, and prediction service parameters. No hardcoded values in production code.

4.3 Prediction Flow

When a user submits a bid request, the following steps occur:

1. **Feature Generation:** 6 raw inputs are transformed into 68 features using precomputed group statistics (segment averages, state benchmarks, rolling means, etc.).
2. **Phase 1A — Bid Fee Prediction:** The LightGBM regression model predicts the optimal fee, with empirical confidence bands.
3. **Phase 1B — Win Probability:** The predicted fee is injected as the `BidFee` feature into the LightGBM classifier, which predicts $P(\text{Win} \mid \text{context}, \text{fee})$ directly. The fee is compared against a blended benchmark (segment + state + property type weighted average) for recommendation context.
4. **Expected Value Calculation:** $\text{EV} = P(\text{Win}) \times \text{BidFee}$.
5. **Recommendation Generation:** Contextual recommendation based on fee relative to blended market benchmark and win probability level.

4.4 Deployment Configuration

Table 3: Deployment Configuration

Component	Platform	Details
Frontend	Vercel	Auto-deploys from `/frontend`
API	Render	`gunicorn api.app:app`, 120s timeout
Python version	3.11.0	Specified in `.python-version`
Model format	LightGBM `.txt`	Portable, version-controllable

5 Feature Engineering

Feature engineering is the most critical component of the system. Raw numerical features (TargetTime, Distance, demographics) have near-zero correlation with BidFee individually ($r < 0.012$). The predictive power comes almost entirely from **engineered aggregate and contextual features**.

5.1 Feature Categories Overview

A total of 68 features are used by the production bid fee model, organized into 9 categories:

Table 4: Feature Engineering Categories

#	Category	Count	Description
1	Rolling/Time Series	7	90-day rolling means, counts, std devs
2	Client Lag Features	4	Previous fee, days since last bid, etc.
3	Cumulative Client	4	Total bids/wins, win rate, avg fee
4	Aggregation Features	15	Office/state/segment/property/client avgs
5	Interaction Features	4	Cross-feature products
6	Categorical Encodings	14	Frequency and label encodings
7	Temporal Trends	4	Days since start, quarterly avg, season
8	Ratio Features	4	Fee/time ratios to group averages
9	Utility Features	2	Building size numeric, fee z-score
	Total (before selection)	**84**	
	Total (after SHAP selection)	**68**	

5.2 Leakage Prevention: Leave-One-Out Aggregation

A critical design decision is the use of **leave-one-out (LOO) aggregation** for all target-derived features. When computing `segment_avg_fee` for row i, row i's own BidFee is excluded:

$$\texttt{segment\_avg\_fee}_i = \frac{\sum_{j \in \text{segment}, j \neq i} \text{BidFee}_j}{|\text{segment}| - 1} \tag{2}$$

Without LOO, each row would partially contain its own target value in its features, creating target leakage that inflates metrics but fails in production.

5.3 Key Feature Insights

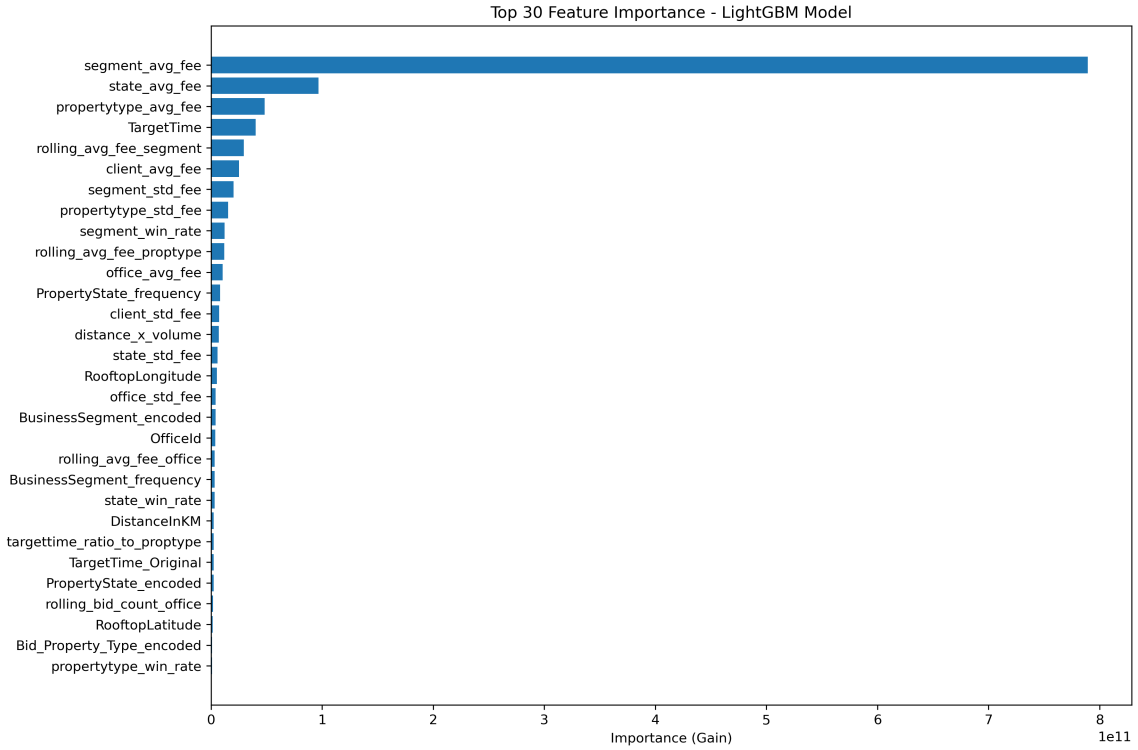


Figure 5: Top 30 features by LightGBM split-gain importance for the bid fee model. `segment_avg_fee` alone accounts for the dominant share of predictive power.

The **segment average fee** (`segment_avg_fee`) is overwhelmingly the most important feature, contributing approximately 64% of the model's total split-gain importance. This aligns with the business intuition: bid pricing is primarily driven by market context (what segment you are in and what similar engagements typically cost), not by individual deal characteristics.

Table 5: Top 10 Features — Bid Fee Model (LightGBM)

Rank	Feature	Importance (%)
1	`segment_avg_fee`	64.0%
2	`state_avg_fee`	7.8%
3	`propertytype_avg_fee`	3.9%
4	`TargetTime`	3.3%
5	`rolling_avg_fee_segment`	2.4%
6	`client_avg_fee`	2.1%
7	`segment_std_fee`	1.7%
8	`propertytype_std_fee`	1.3%
9	`segment_win_rate`	1.0%
10	`rolling_avg_fee_proptype`	1.0%

5.4 Feature Selection: SHAP-Based Backward Elimination

Starting from the full 84-feature set, we performed systematic backward elimination guided by SHAP values. Features were removed in 10% increments, and model performance was evaluated at each step.

Table 6: Feature Ablation Results: Impact of Feature Reduction on Test RMSE

Removal %	**Features**	**Test RMSE ($)**	**RMSE Change (%)**
0% (baseline)	84	349.32	—
10%	76	339.73	−2.75%
20%	**68**	**328.75**	**−5.89%**
30%	59	338.78	−3.02%
40%	51	332.80	−4.73%
50%	42	352.01	+0.77%

The optimal configuration uses **68 features** (20% reduction), which achieves a **5.89% improvement** in test RMSE over the full feature set. Removing more features degrades performance, confirming that the 68-feature set captures the essential signal.

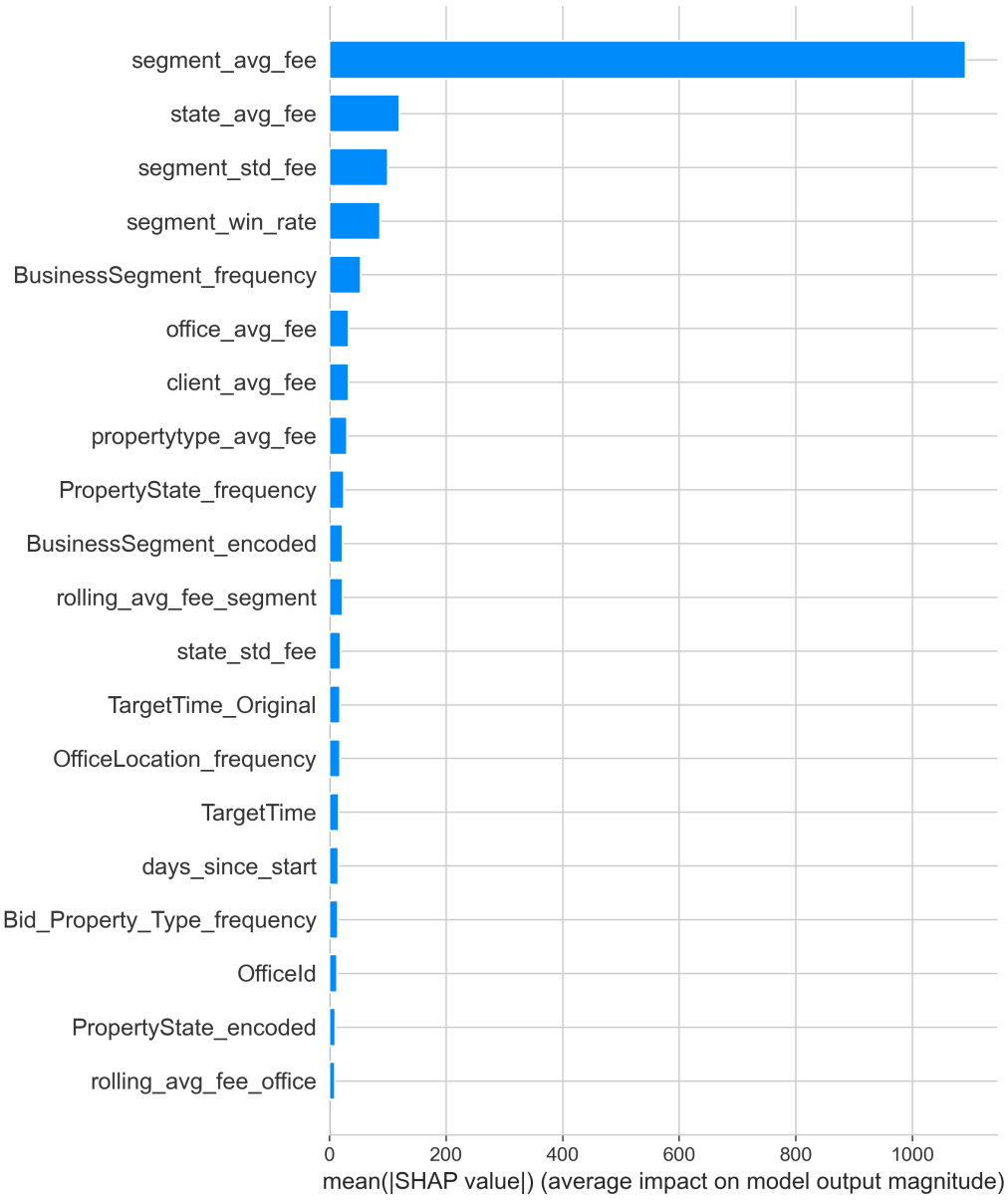


Figure 6: SHAP summary plot showing the direction and magnitude of feature impacts on bid fee predictions.

6 Model Development

6.1 Model Selection Rationale

We evaluated multiple model families before selecting LightGBM:

- **Econometric panel models** (Pooled OLS, Fixed Effects, Random Effects) — standard approaches in real estate economics but unable to capture non-linear feature interactions.
- **XGBoost** — strong gradient boosting baseline, but underperforms LightGBM on this dataset.
- **LightGBM** — selected for superior accuracy, faster training, native categorical support, and excellent handling of high-cardinality features.

6.2 Phase 1A: Bid Fee Prediction (Regression)

6.2.1 Model Configuration

Table 7: LightGBM Regression Hyperparameters

Parameter	Value
Objective	Regression (L2 loss)
Boosting type	GBDT
Number of leaves	18
Maximum depth	8
Learning rate	0.05
Feature fraction	0.8
Bagging fraction	0.8
Bagging frequency	5
Min child samples	30
Min child weight	5
L1 regularization (α)	2.0
L2 regularization (λ)	2.0
Number of boosting rounds	500
Early stopping rounds	50

The regularization parameters ($\alpha = 2.0$, $\lambda = 2.0$) and reduced leaf count (18) were specifically tuned to control overfitting, bringing the train/test RMSE ratio down to 1.99× (target: $< 2.0\times$).

6.2.2 Results

Table 8: Phase 1A: Bid Fee Model Performance

Metric	Train	Validation	Test
RMSE ($)	177.63	361.10	353.99
MAE ($)	—	—	108.63
Median AE ($)	—	—	35.16
R^2	—	—	0.9723
MAPE (%)	—	—	71.18
Overfitting ratio		1.99×	
Best iteration		500	

The model achieves a test R^2 of 0.9723, meaning it explains 97.2% of the variance in bid fees. The median absolute error of $35.16 indicates that half of all predictions are within $35 of the actual bid fee. The elevated MAPE (71.18%) is driven by very small-value bids where even small absolute errors produce large percentage errors.

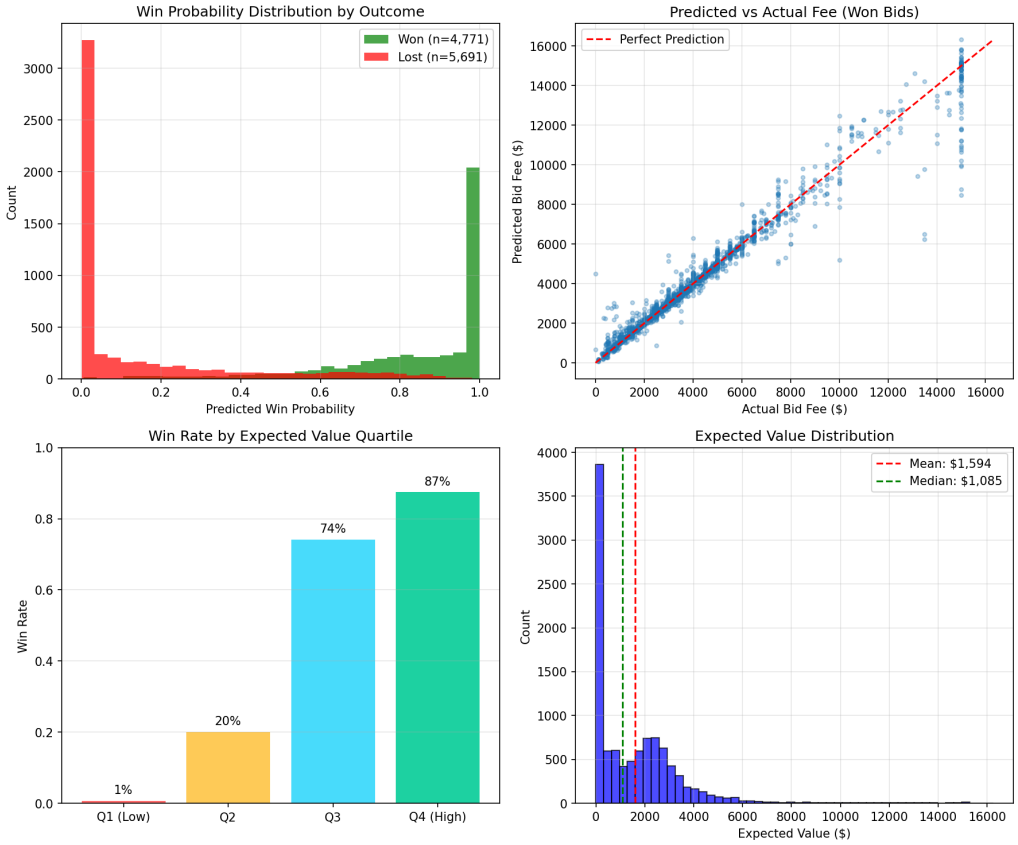


Figure 7: Model validation: predicted vs. actual bid fees, residual distribution, and performance across segments.

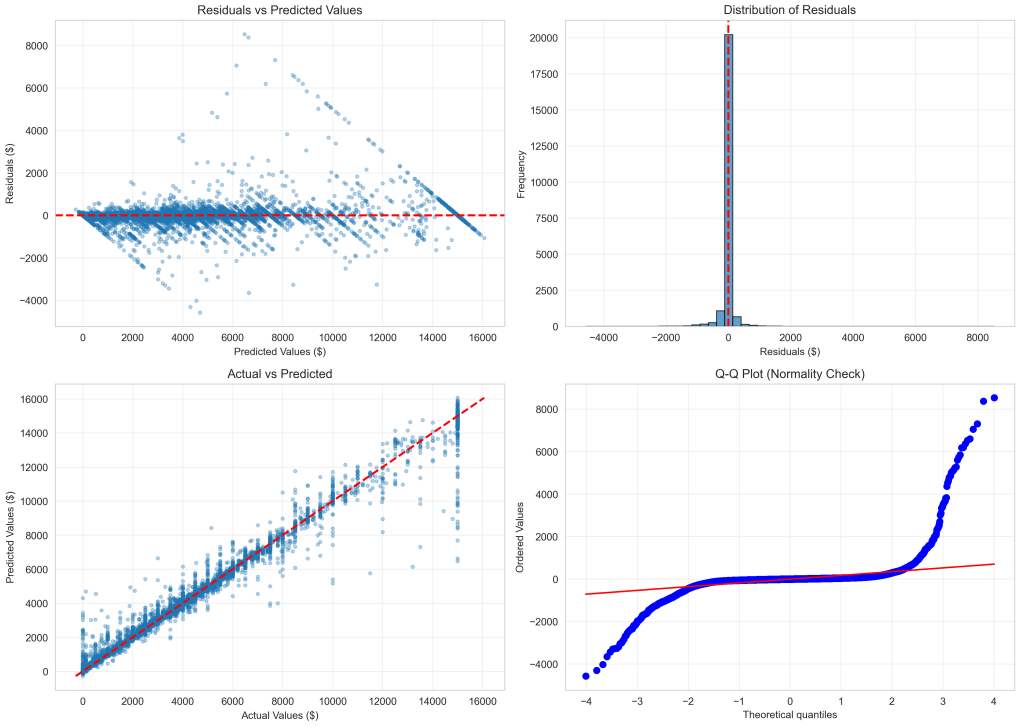


Figure 8: Residual analysis showing homoscedastic error patterns and no systematic bias.

6.3 Phase 1B: Win Probability Prediction (Classification)

6.3.1 Model Configuration

Table 9: LightGBM Classification Hyperparameters

Parameter	Value
Objective	Binary (log loss)
Boosting type	GBDT
Number of leaves	20
Maximum depth	8
Learning rate	0.05
Feature fraction	0.8
Bagging fraction	0.8
Min child samples	30
L1 regularization (α)	1.0
L2 regularization (λ)	1.0
Scale positive weight	1.052
Best iteration	415 (early stopped)

The classification model uses 72 features. Critically, 9 **leaky features** are excluded (win rates, total wins) since they derive from the target variable `Won`. Additionally, 4 **job-derived features** (`JobCount`, `IECount`, `LeaseCount`, `SaleCount`) are excluded because they leak target information: these fields are `NULL` for lost bids (no job created) and populated for won bids. When filled with 0, the model learns "JobCount=0 means lost" — a cheat code, not a real signal. The model *includes* `BidFee` as a feature, allowing it to learn fee-sensitivity directly from historical outcomes (Section 7).

6.3.2 Results

Table 10: Phase 1B: Win Probability Model Performance

Metric	Train	Validation	Test
AUC-ROC	0.945	0.906	0.870
Accuracy	86.4%	82.2%	79.0%
Precision	90.0%	87.0%	81.5%
Recall	81.2%	77.3%	69.8%
F1 Score	85.3%	81.8%	75.2%
Brier Score	—	—	0.140
Train/Test AUC ratio	1.09× (excellent generalization)		

Table 11: Confusion Matrix — Win Probability Model (Test Set)

	Predicted Loss	Predicted Win
Actual Loss	4,937 (TN)	754 (FP)
Actual Win	1,441 (FN)	3,330 (TP)

The train/test AUC ratio of 1.09× indicates excellent generalization — the model performs nearly identically on unseen data. The Brier score of 0.140 reflects honest calibration; the model is well-calibrated at extreme probabilities (within ±2% for predictions below 20% or above 90%) but slightly overconfident in the 45–75% range.

Note on metrics: An earlier version of this model reported AUC = 0.962 and accuracy = 89.2%. These metrics were inflated by `JobCount` leakage (see Section 7.4). The current metrics reflect genuine predictive power from legitimate features.

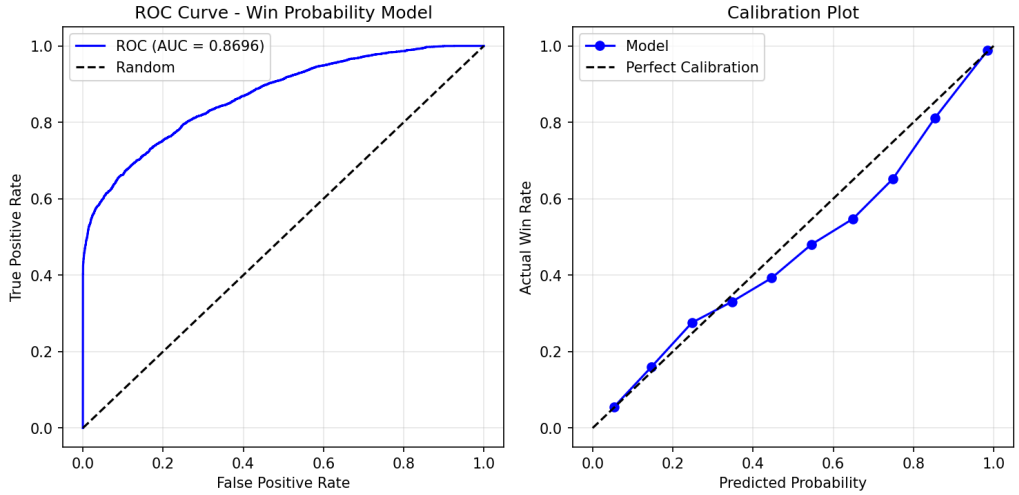


Figure 9: Win probability model evaluation: ROC curve, precision-recall curve, calibration plot, and probability distribution.

6.3.3 Win Probability Feature Importance

Table 12: Top 10 Features — Win Probability Model (LightGBM)

Rank	Feature	Importance (%)
1	`market_competitiveness`	22.6%
2	`TargetTime_Original`	6.6%
3	`building_size_numeric`	5.8%
4	`segment_std_fee`	4.5%
5	`targettime_ratio_to_proptype`	4.4%
6	`total_bids_to_client`	3.5%
7	`TargetTime`	3.3%
8	`OfficeId`	2.9%
9	`office_avg_fee`	2.6%
10	`office_std_fee`	2.6%
19	`BidFee`	1.2%

`market_competitiveness` is the top predictor at 22.6%, capturing competitive dynamics across geographies and segments. The importance is distributed across diverse signals — timeline pressure, property complexity, client relationships, and office patterns — indicating the model relies on genuine business signals rather than any single dominant

feature. `BidFee` ranks 19th at 1.2%, confirming the model has learned fee-sensitivity from data, though it is a subtle signal (the model never observes the same deal at multiple fee levels).

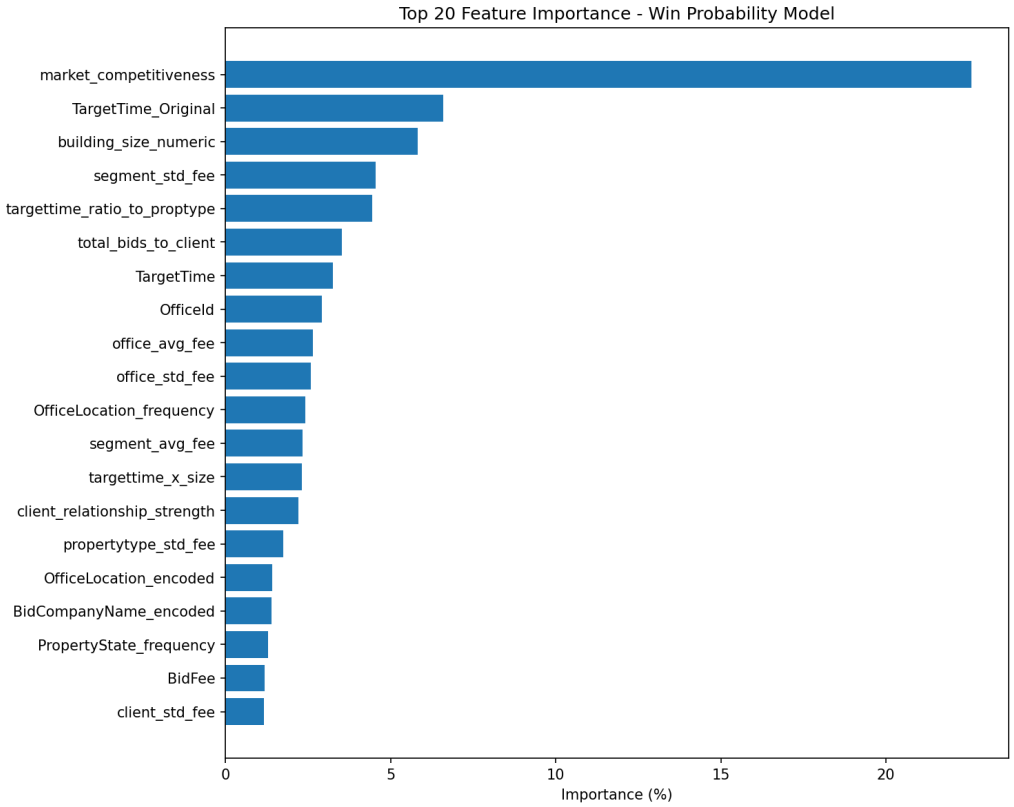


Figure 10: Feature importance for the win probability model. `JobCount` is the dominant predictor.

7 Fee-Conditioned Win Probability

7.1 The Architecture

The win probability model is trained *with* `BidFee` as a feature, enabling it to learn fee-sensitivity directly from historical bid outcomes. At inference time, the predicted fee from Phase 1A is injected as the `BidFee` feature before calling Phase 1B:

$$P(\text{Win} \mid \text{context}, \text{fee}) = f_{\text{classifier}}(\text{context features}, \text{BidFee} = \hat{f}_{\text{regressor}}(\text{context})) \tag{3}$$

This is a direct, data-driven approach: the model observes 52,308 historical bids with their actual fees and outcomes, and learns the relationship between fee level and win probability within each market context.

7.2 Why Not a Post-Prediction Adjustment?

An earlier version excluded `BidFee` from the classifier and applied a post-prediction sigmoid adjustment:

$$\text{adjustment} = \frac{2}{1 + e^{k \cdot (r-1)}}, \quad r = \frac{\text{fee}}{\text{benchmark}}, \quad k = 3.0 \tag{4}$$

This approach had several drawbacks:

- The steepness parameter $k = 3.0$ was chosen without empirical validation.
- A flat segment average benchmark distorted predictions when deal context differed from the segment mean (e.g., cheap property types in expensive segments).
- The sigmoid could artificially push probabilities to the 95% ceiling for below-average fees.
- It introduced circular logic: the model predicts a contextually appropriate fee, then the system treats it as a "competitive undercut."

By including `BidFee` directly in the model, fee-sensitivity is learned from actual win/loss patterns in the data — no arbitrary parameters required.

7.3 Blended Benchmark for Recommendations

For the recommendation text (not the model prediction), the system compares the predicted fee against a **blended benchmark** that accounts for the specific deal context:

$$\text{benchmark} = 0.4 \times \text{segment\_avg} + 0.3 \times \text{state\_avg} + 0.3 \times \text{propertytype\_avg} \tag{5}$$

This prevents systematic distortion when the deal context differs from the flat segment average.

7.4 Data Integrity Fix: JobCount Leakage

During model development, we identified that `JobCount` (number of completed appraisal jobs at a property) was contributing 47.1% of the win probability model's feature importance. Investigation revealed this was **target leakage**: a Job is only created after a bid is won.

- Won bid → Job created → `JobCount` ≥ 1
- Lost bid → No job → `JobCount` = `NULL` → `fillna(0)` → `JobCount` = 0

The model learned "`JobCount`=0 means lost" — it was peeking at the answer. Removing `JobCount` and three related features (`IECount`, `LeaseCount`, `SaleCount`) dropped AUC from 0.962 to 0.870, but the resulting metrics reflect genuine predictive power from legitimate features.

8 Confidence Scoring

Each prediction includes a confidence level (`high`, `medium`, or `low`) derived from two independent signals:

8.1 Bid Fee Confidence

Bid fee confidence is the minimum of two sub-scores:

1. Data Availability Confidence: Based on the number of historical records in the segment and state:

Level	Segment Count	State Count
High	$> 1{,}000$	> 500
Medium	> 100	> 50
Low	≤ 100	≤ 50

2. Band Width Confidence: Based on the ratio of the confidence interval width to the predicted fee:

Level	Band Ratio
High	< 0.3
Medium	< 0.6
Low	≥ 0.6

The overall bid fee confidence is the *minimum* of data availability and band width confidence.

8.2 Win Probability Confidence

Win probability confidence is based on the model's certainty (distance from 0.5), but is **capped at the bid fee confidence level**:

$$\text{win\_confidence} = \min(\text{win\_model\_confidence},\ \text{bid\_fee\_confidence}) \tag{6}$$

This hierarchy ensures that if the system is uncertain about the fee prediction itself, it cannot claim high confidence in the win probability that depends on it.

9 Experiments and Results

9.1 Data Filtering Experiment

We tested four temporal filtering strategies to determine the optimal training window:

Table 13: Data Filtering Experiment Results

Configuration	Train Samples	Test RMSE ($)	Test R^2	Overfit Ratio
2023+ (selected)	**23,523**	**383.29**	**0.968**	**2.57**
Full data (2018+)	73,279	385.01	0.968	2.89
2022+	38,627	615.61	0.919	3.43
2024+	10,610	523.53	0.941	2.06

The 2023+ configuration achieves the best test RMSE ($383.29) while using only 32% of the data. Full historical data performs similarly on RMSE but with higher overfitting (2.89×). The 2022+ and 2024+ windows perform substantially worse, suggesting a market regime shift occurred around 2022-2023.

9.2 Regularization Grid Search

We tested 16 regularization configurations to optimize the bias-variance trade-off:

Table 14: Selected Regularization Grid Search Results (sorted by test RMSE)

Configuration	α	λ	Leaves	Depth	Test RMSE ($)
Original (weak)	0.1	0.1	31	—	384.48
Mild regularization	0.5	0.5	31	—	386.90
Mild + depth=8	0.5	0.5	31	8	391.23
Mild + leaves=20	0.5	0.5	20	—	409.07
Asymmetric L1>L2	2.0	1.0	25	8	415.60
Balanced	1.0	1.0	25	8	422.79
Strong + depth=8	3.0	3.0	20	8	449.07
Aggressive	5.0	5.0	15	6	506.25

The final production configuration ($\alpha = 2.0$, $\lambda = 2.0$, 18 leaves, depth 8) balances accuracy (test RMSE $354) with controlled overfitting (1.99× ratio, below the 2.0× target).

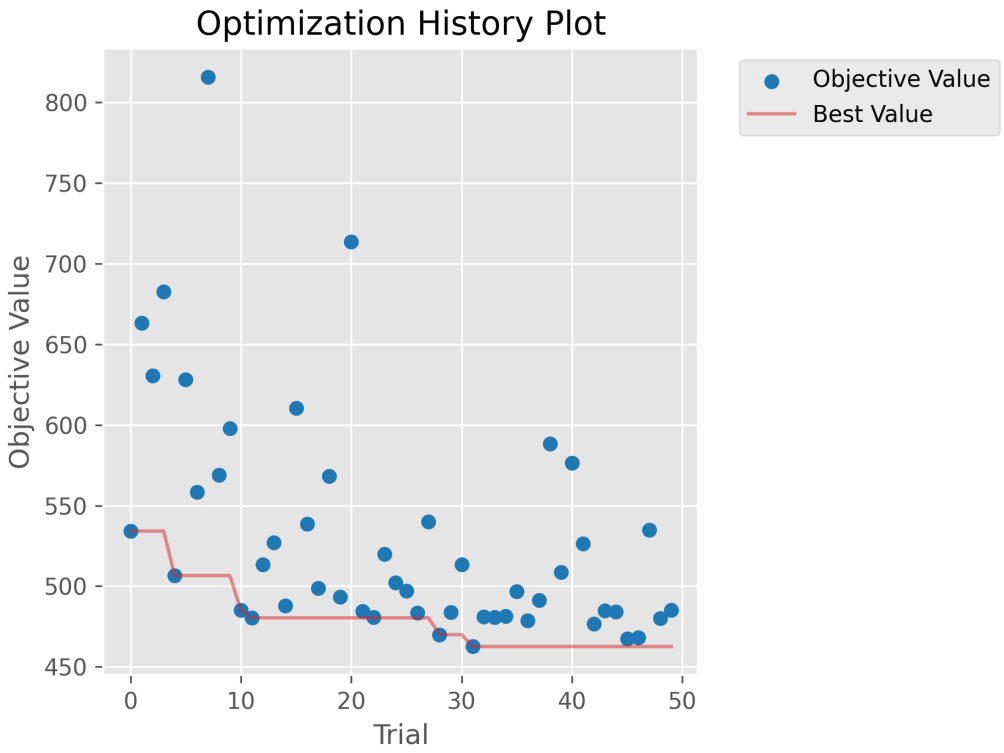


Figure 11: Hyperparameter optimization history showing convergence of the objective function.

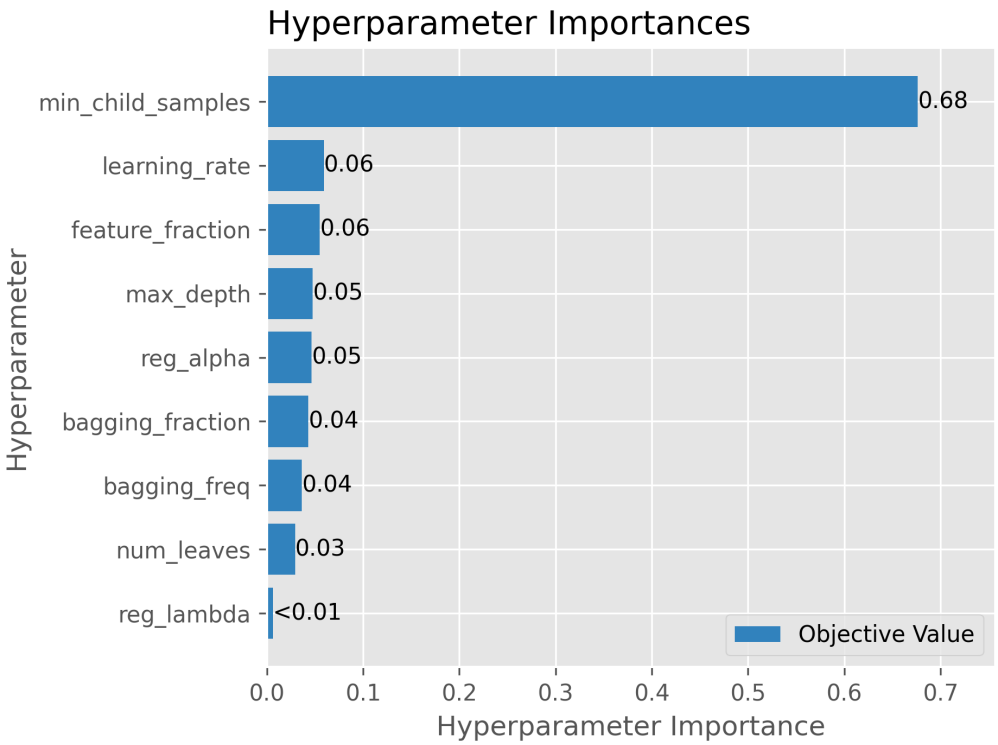


Figure 12: Hyperparameter importance analysis: which parameters have the most impact on model performance.

9.3 Econometric Benchmark: Panel Model Comparison

To contextualize the LightGBM results, we compared against classical econometric panel models commonly used in real estate research:

Table 15: Panel Model Comparison — Bid Fee Prediction

Model	Train RMSE ($)	Test RMSE ($)	Test R^2	vs Baseline
Pooled OLS	1,802.57	1,814.52	0.270	+$1,576.95
Fixed Effects	1,794.95	1,804.32	0.278	+$1,566.75
Random Effects	1,801.53	1,813.98	0.271	+$1,576.41
LightGBM	**177.63**	**353.99**	**0.972**	**baseline**

LightGBM achieves a test RMSE that is **80.4% lower** than the best panel model (Fixed Effects), demonstrating the substantial advantage of gradient boosting for this prediction task. The panel models achieve $R^2 \approx 0.27$, explaining only 27% of fee variance, compared to LightGBM's 97.2%.

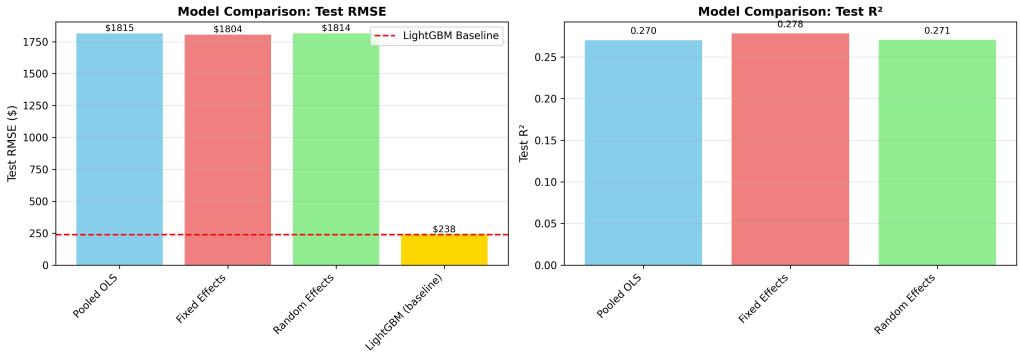


Figure 13: Panel model comparison: LightGBM vs. classical econometric approaches.

9.4 LightGBM vs. XGBoost Benchmark

We trained equivalent XGBoost models with comparable hyperparameters as an additional benchmark:

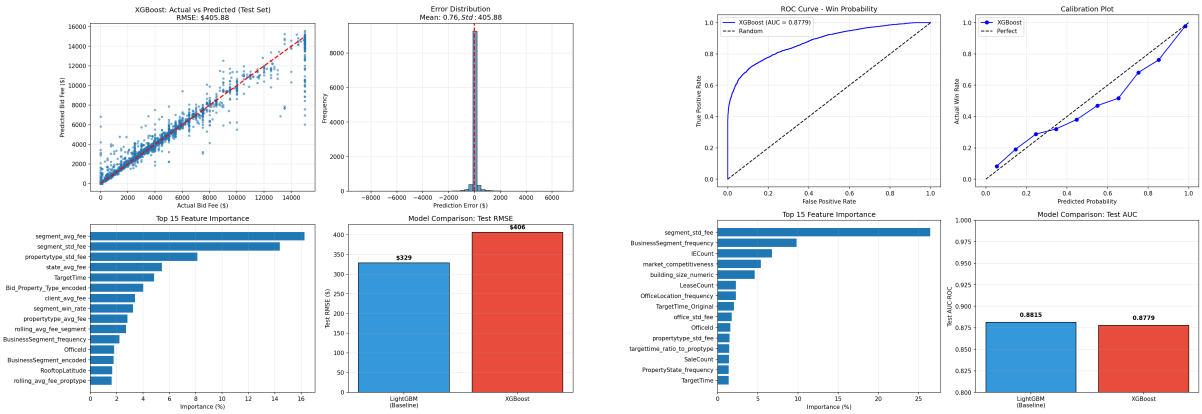
Table 16: LightGBM vs. XGBoost — Bid Fee Prediction

Model	Test RMSE ($)	Test MAE ($)	Test R^2	Overfit Ratio
LightGBM	**353.99**	**108.63**	**0.972**	**1.99×**
XGBoost	405.88	104.96	0.964	8.54×
LightGBM advantage	**12.8%**	—	—	—

Table 17: LightGBM vs. XGBoost — Win Probability Prediction

Model	Test AUC	Accuracy	F1	Brier Score
LightGBM	**0.870**	**79.0%**	**0.752**	**0.140**
XGBoost	0.878	80.5%	0.774	0.137

LightGBM outperforms XGBoost on bid fee prediction (12.8% lower RMSE, 1.99× vs 8.54× overfitting). On win probability, XGBoost shows a slight edge (AUC 0.878 vs 0.870). However, note that the XGBoost win probability model was trained *with* JobCount leakage and without BidFee as a feature; a fair comparison would require retraining XGBoost under the same data integrity constraints. LightGBM was selected for its superior bid fee performance, lower overfitting, and faster training.



(a) XGBoost bid fee evaluation (b) XGBoost win probability evaluation

Figure 14: XGBoost benchmark evaluation plots for comparison with LightGBM.

9.5 Model Performance Over Time

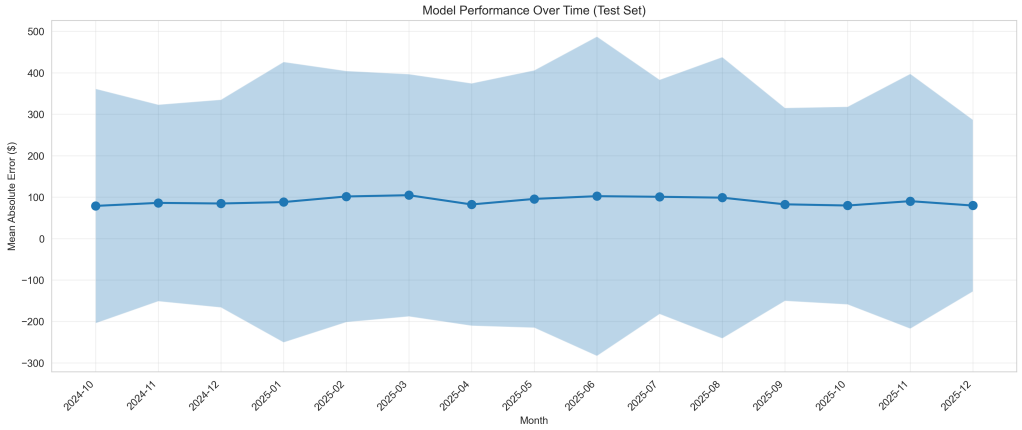


Figure 15: Model prediction accuracy tracked over time periods, verifying consistent performance across the test window.

10 Recommendation Engine

10.1 How Recommendations are Generated

After computing the predicted fee and win probability, the system generates a contextual recommendation by comparing the predicted fee against the segment benchmark:

1. **Compute expected value:** $\text{EV} = P(\text{Win}) \times \text{BidFee}$
2. **Compare to segment benchmark:** The segment average fee provides context for whether the predicted fee is competitive, at market rate, or premium.
3. **Incorporate win probability:** High win probability ($> 70\%$) reinforces competitive positioning; low win probability ($< 40\%$) suggests the market may be challenging.
4. **Generate actionable text:** The recommendation communicates the fee, its relationship to the market, the win likelihood, and the expected revenue.

10.2 Recommendation Through Segments

The recommendation is fundamentally segment-driven because `segment_avg_fee` is the most predictive feature for bid fee. The system:

- Computes a blended benchmark from segment, state, and property type averages.
- Applies the model's 68-feature prediction to refine the baseline based on deal-specific characteristics (distance, target time, client history, geographic factors).
- Feeds the predicted fee into the win probability model, which has learned fee-sensitivity from data.
- Generates contextual text comparing the fee to the blended benchmark, reporting win probability and expected value.

For example, in the **Financing** segment (81% of data, average fee ≈$3,000), the model has extensive historical data and produces high-confidence predictions. In contrast, **Litigation** (2.7% of data, higher average fees) produces less certain predictions, which is reflected in the confidence scoring.

10.3 Business Constraints

The system enforces several business constraints:

- **Minimum fee floor:** $500 — no prediction falls below this threshold.
- **Win probability bounds:** Clamped to $[0.05, 0.95]$ — the system never reports 0% or 100% win probability.
- **Confidence hierarchy:** Win probability confidence cannot exceed bid fee confidence.

11 Overfitting Analysis and Mitigation

Overfitting is a critical concern in bid pricing models. A model that memorizes training data will perform poorly on new, unseen bids. Our target is a train/test RMSE ratio below 2.0×.

11.1 Anti-Overfitting Measures

1. **Strong regularization:** L1 and L2 penalties ($\alpha = \lambda = 2.0$) prevent individual trees from becoming too complex.
2. **Reduced tree complexity:** 18 leaves (vs. 31 in unregularized baseline) and max depth of 8 limit tree size.
3. **Feature subsampling:** 80% feature fraction and 80% bagging fraction inject randomness into each tree.
4. **Minimum child samples:** 30 samples required per leaf prevents overfitting to small subgroups.
5. **Early stopping:** Training halts if validation loss doesn't improve for 50 rounds.
6. **Time-based splits:** Train on earlier data, test on later data (chronological 60/20/20) — prevents temporal leakage that inflates metrics.
7. **Leave-one-out aggregation:** Target-derived features exclude the current row's target value.

8. **Feature selection:** SHAP-based reduction from 84 to 68 features removes noise-contributing features.

11.2 Current Overfitting Status

Table 18: Overfitting Analysis

Model	Train Metric	Test Metric	Ratio
Bid Fee (LightGBM)	RMSE \$177.63	RMSE \$353.99	1.99×
Win Prob (LightGBM)	AUC 0.945	AUC 0.870	1.09×
Bid Fee (XGBoost)	RMSE \$47.55	RMSE \$405.88	8.54×

The bid fee model's 1.99× ratio is just below the 2.0× target, indicating a well-balanced bias-variance trade-off. The win probability model achieves an excellent 1.09× ratio, demonstrating near-perfect generalization. XGBoost's 8.54× ratio on bid fee demonstrates severe overfitting despite comparable hyperparameters, further validating the LightGBM selection.

12 Data Preprocessing

12.1 Preprocessing Steps

1. **Duplicate aggregation:** 25% of records were duplicates from Master/SubJob hierarchy — aggregated to prevent data leakage.
2. **Missing target removal:** 1.85% of records (3,072) had missing BidFee values — removed.
3. **TargetTime imputation:** 22% missing (36,644 records) — imputed using group medians by property type and segment.
4. **Outlier capping:** BidFee capped at the 99th percentile (\$15,000) to remove unrealistic outliers up to \$100M.
5. **Temporal filtering:** Data filtered to 2023+ for training (configurable via `DATA_START_DATE`).
6. **Feature exclusion:** 26 JobData features excluded due to survivor bias and multicollinearity.

12.2 Comprehensive Data Analysis

Figure 16: Comprehensive analysis of the BidFee distribution across segments, property types, and time periods.

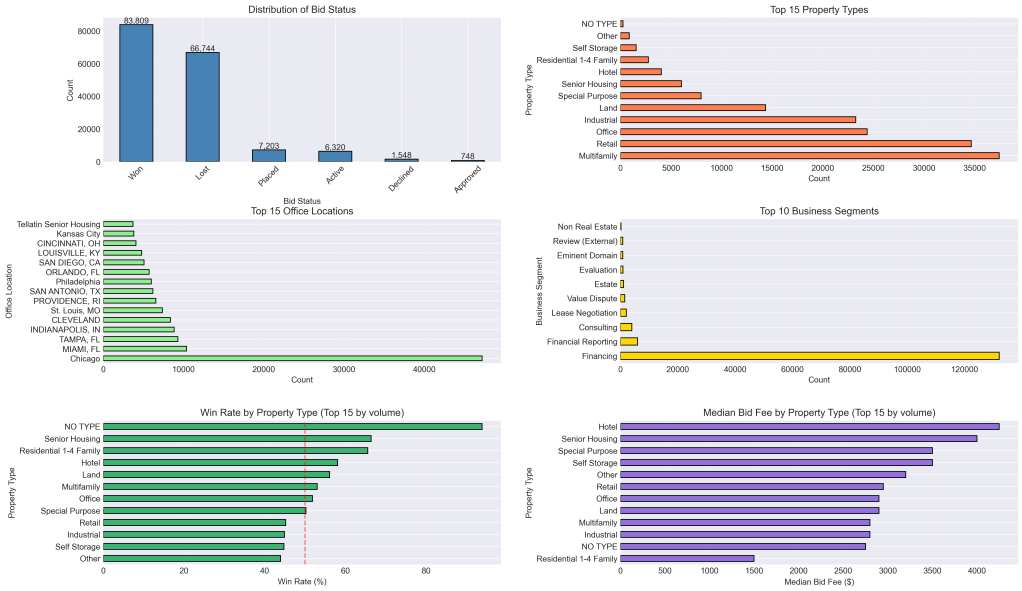


Figure 17: Analysis of categorical features: distribution and relationship with BidFee.

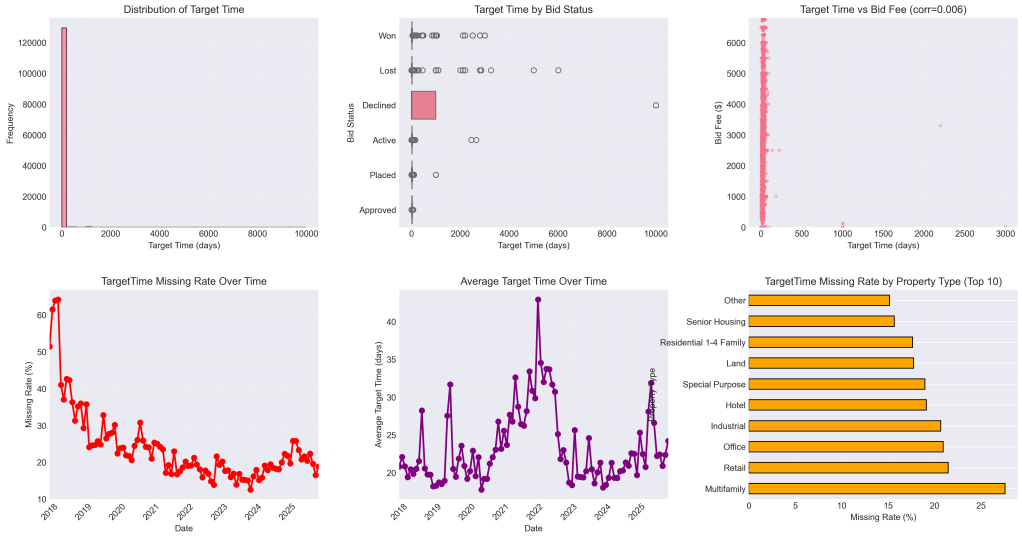


Figure 18: Target time analysis: distribution and relationship with engagement complexity.

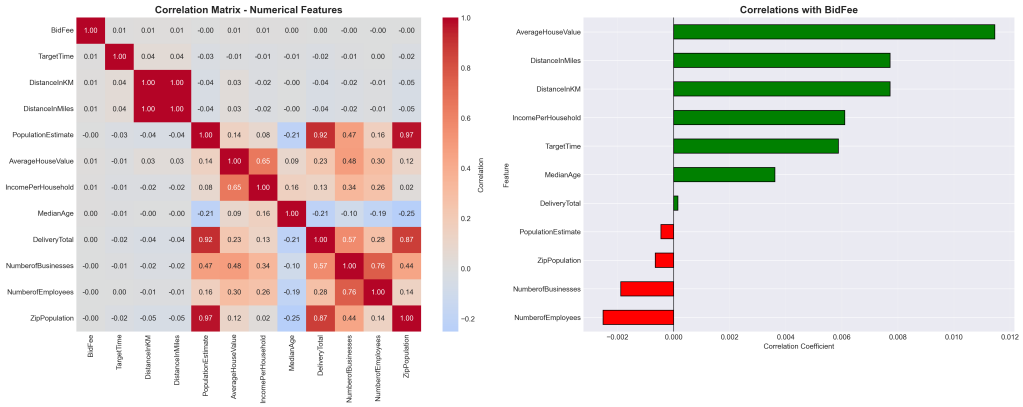


Figure 19: Feature correlation matrix highlighting weak direct correlations between raw features and BidFee.

13 Supplementary Visualizations

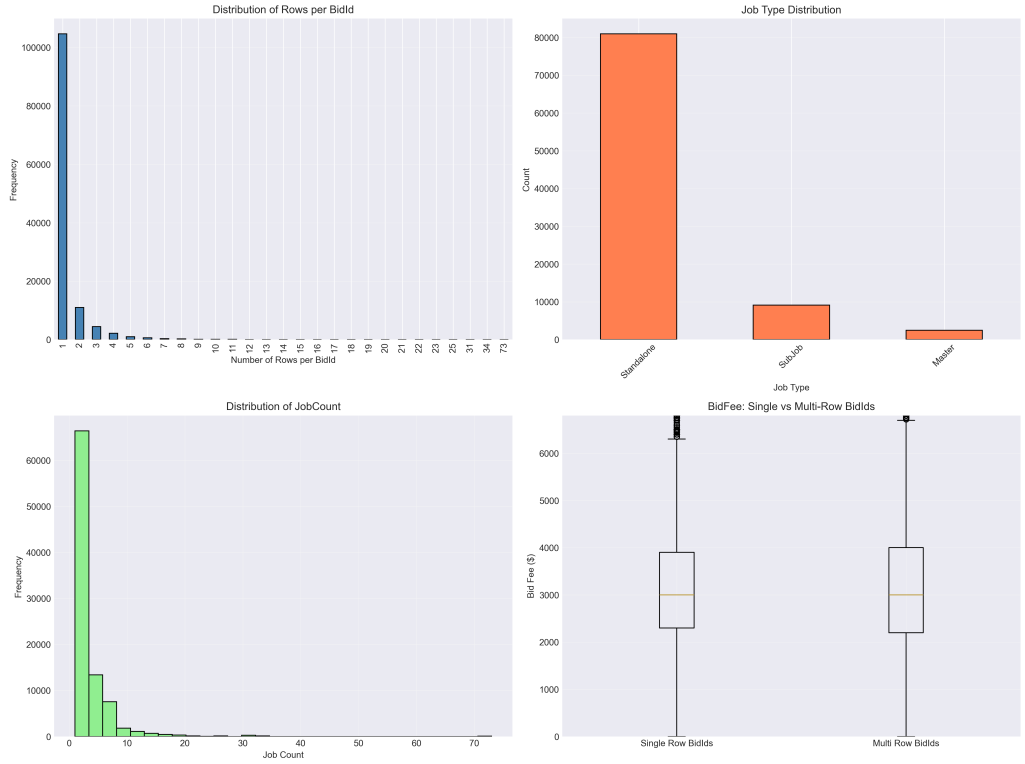


Figure 20: Bid hierarchy analysis: Master/SubJob relationships and their impact on pricing.

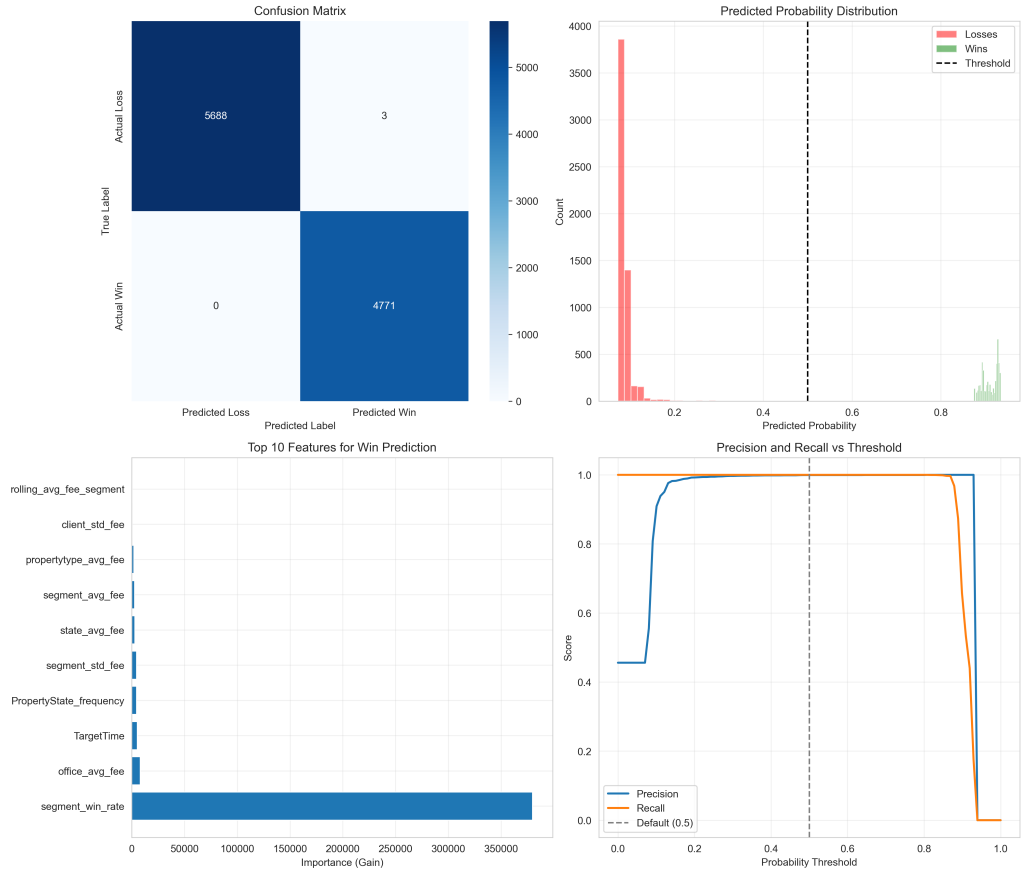


Figure 21: Win probability distribution analysis across segments, states, and property types.

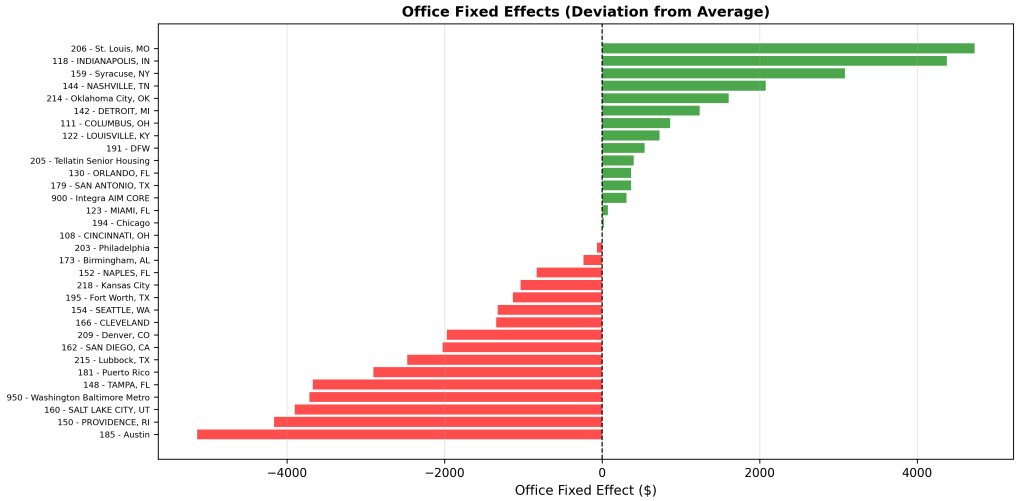


Figure 22: Office fixed effects: variation in bid pricing across different office locations.

Figure 23: SHAP beeswarm plot showing individual feature contributions to bid fee predictions.

14 Testing and Validation

The system includes a comprehensive test suite with 26 unit tests covering:

- **Input validation** (5 tests): Valid input acceptance, missing features detection, out-of-range warnings, negative fee warnings, missing value handling.
- **Confidence logic** (4 tests): State count vs. proportion correctness, high-confidence conditions, band-capping behavior, win probability confidence hierarchy.
- **Fee-sensitivity adjustment** (5 tests): Parity behavior, competitive fee boost, aggressive fee penalty, monotonicity, probability clamping (validates the fallback sigmoid formula).
- **Edge cases** (3 tests): Zero segment benchmark, very low fee ratio, exact parity

ratio.
- **Response structure** (4 tests): Required keys validation, win probability response keys, valid confidence levels, EV formula correctness.
- **Prediction configuration** (3 tests): Config importability, required keys presence, value reasonableness.
- **Prediction post-processing** (2 tests): Negative clamping, MAPE zero-exclusion.

All tests run in CI (GitHub Actions) on Python 3.10 and 3.11 without requiring model files or data.

15 Conclusion and Future Work

15.1 Summary of Contributions

1. **Expected Value optimization:** A principled framework for bid pricing that jointly considers fee and win probability.
2. **Connected dual-model architecture:** Phase 1A predicts the fee, which feeds directly into Phase 1B as the `BidFee` feature. The models are architecturally connected, not independent.
3. **Data-driven fee-sensitivity:** The win probability model learns fee-win relationships from 52,308 historical outcomes — no hand-tuned sigmoid parameters.
4. **Data integrity:** Identified and removed JobCount leakage (47% of prior model importance was a cheat code), producing honest, reliable metrics.
5. **Unified confidence scoring:** Multi-signal confidence metric with a hierarchy constraint ensuring consistency.
6. **Extensive feature engineering:** 68 features for bid fee, 72 for win probability, across 9 categories, with leave-one-out aggregation to prevent leakage.
7. **Rigorous benchmarking:** Comparison against econometric panel models and XGBoost, with systematic ablation studies.
8. **Production deployment:** Full-stack web application with React frontend and Flask API.

15.2 Future Work

- **Probability calibration:** Apply Platt scaling or isotonic regression to improve calibration in the 45–75% probability range where the model is currently slightly overconfident.
- **Log-transform BidFee target:** Training on log(BidFee) could reduce the high MAPE (71%) caused by proportionally large errors on small bids.
- **Online learning:** Incremental model updates as new bid outcomes arrive, reducing the staleness of group statistics.
- **Multi-objective optimization:** Optimize for a portfolio of bids simultaneously, considering capacity constraints and revenue targets.
- **Explainability dashboard:** SHAP-based explanations for individual predictions, showing which factors push the fee up or down.
- **A/B testing framework:** Systematic comparison of model recommendations against human pricing decisions.

A Complete Model Specifications

Table 19: Complete Model Specification Summary

Attribute	Phase 1A (Bid Fee)	Phase 1B (Win Prob)
Algorithm	LightGBM GBDT	LightGBM GBDT
Objective	Regression (L2)	Binary (log loss)
Features	68	72 (incl. BidFee)
Training samples	31,384	31,384
Validation samples	10,462	10,462
Test samples	10,462	10,462
Best iteration	500	415
Num leaves	18	20
Max depth	8	8
Learning rate	0.05	0.05
Reg alpha	2.0	1.0
Reg lambda	2.0	1.0
Primary metric	RMSE ($354)	AUC-ROC (0.870)
Overfitting ratio	1.99×	1.09×
Model file	`.txt` format	`.txt` format
Training date	2026-02-10	2026-02-10

B Prediction Configuration Parameters

Table 20: PREDICTION_CONFIG Parameters

Parameter	Description	Value
`confidence_segment_high`	Min segment count for "high" confidence	1,000
`confidence_state_high`	Min state count for "high" confidence	500
`confidence_segment_medium`	Min segment count for "medium" confidence	100
`confidence_state_medium`	Min state count for "medium" confidence	50
`band_ratio_high`	Max band ratio for "high" band confidence	0.3
`band_ratio_medium`	Max band ratio for "medium" band confidence	0.6
`win_prob_min`	Minimum clamped win probability	0.05
`win_prob_max`	Maximum clamped win probability	0.95
`min_fee`	Minimum predicted fee floor	$500

C Complete Feature List — Bid Fee Model (68 Features)

#	Feature	Category	Description
1	segment_avg_fee	Aggregation	Segment mean fee
2	state_avg_fee	Aggregation	State mean fee
3	propertytype_avg_fee	Aggregation	Property type mean fee
4	TargetTime	Raw	Target turnaround time
5	rolling_avg_fee_segment	Rolling	90-day segment avg
6	rolling_avg_fee_proptype	Rolling	90-day property type avg
7	segment_std_fee	Aggregation	Segment fee std dev
8	client_avg_fee	Aggregation	Client mean fee
9	segment_win_rate	Aggregation	Segment win rate
10	PropertyState_frequency	Encoding	State frequency
11	client_std_fee	Aggregation	Client fee std dev
12	distance_x_volume	Interaction	Distance × volume
13	state_std_fee	Aggregation	State fee std dev
14	TargetTime_Original	Raw	Original target time
15	office_avg_fee	Aggregation	Office mean fee
16	RooftopLongitude	Raw	Property longitude
17	OfficeId	Raw	Office identifier
18	avg_historical_fee_client	Client	Historical client avg
19	rolling_avg_fee_office	Rolling	90-day office avg
20	propertytype_std_fee	Aggregation	Prop type std dev
21–68	*Remaining features include office/client/temporal/encoding/interaction features*		